

MIXED VARIABLE NON-LINEAR OPTIMIZATION BY DIFFERENTIAL EVOLUTION

Jouni Lampinen

Lappeenranta University of Technology
Department of Information Technology
Laboratory of Information Processing
P.O.Box 20, FIN-53851 Lappeenranta, Finland
phone: +358-5-6243430, fax: +358-5-6243456
E-mail: Jouni.Lampinen@lut.fi

Ivan Zelinka

Technical University of Brno
Faculty of Technology (Zlín)
Department of Automatic Control
nám. T.G.M. 275, 762 72 Zlín, Czech Republic
phone: +420-67-7211521, fax: +420-67-7211521
E-mail: Zelinka@zlin.vutbr.cz

Abstract: *This article discusses solving non-linear programming problems containing integer, discrete and continuous variables. A novel mixed integer-discrete-continuous, non-linear optimization method based on Differential Evolution algorithm is described. Also the required handling techniques for integer, discrete and continuous variables are described including the techniques needed to handle boundary constraints as well as those needed to simultaneously deal with several non-linear and non-trivial constraint functions. Previous experiments and comparisons with other methods for mixed integer-discrete-continuous non-linear optimization have suggested that the described approach is capable of obtaining high quality solutions. The novel method is relatively easy to implement and use. It is found to be effective, efficient and robust, which makes it as an attractive and widely applicable approach for solving practical problems in the field of prediction.*

Keywords: evolutionary algorithms, differential evolution, global optimization, non-linear optimization, mixed discrete variables, penalty function, engineering optimization

1 Introduction

A wide range of problems in the field of prediction can be viewed as optimization problems. For example, optimization of various predictive computer models [Zel98], neural networks [ML97, ZL99b], models based on fractal geometry [ZL99a], etc. These computer models are typically subject to tuning, learning, adapting, fitting, structure design, etc. Surprisingly often these procedures can be formulated as an optimization problem. However, pure optimization approach is often avoided, since the resulting objective function is considered too hard to solve using the existing optimization methods. Thus, optimization of these computer models calls for an efficient, effective and robust non-linear optimization method that is also capable of handling integer and discrete variables, and multiple non-linear constraints when required. Therefore developing a more effective, efficient, robust, easy to use, easy to implement and even more general concepts for non-linear optimization is an important target from prediction field's point of view.

In general, when discussing non-linear programming, the variables of the objective function are usually assumed to be continuous. However, for example in practical engineering optimization the problems in which some or all of the design variables have discrete or integer values are very common. They are commonly discrete because the available values are limited to a set of standard sizes. For example, the thickness of a steel plate, the diameter of a copper pipe, the size of a screw, the module of a gear tooth, the size of a roller bearing, etc., are often limited to a set of commercially available standard sizes. Respectively integer variables are commonly used to express a number of identical elements used in the design. Examples of integer variables are the number of teeth of a gear, the number of bolts or rivets needed to fix a structure, the number of heat exchanger tubes, the number of cooling fins of a heat sink, the number of coils of a spring, the number of nodes or layers of *Neural Network*, etc.

It is clear that in practice, a large fraction of optimization problems fall into the category of *mixed integer-discrete-continuous, non-linear programming* problems. Despite of that, mostly solving continuous problems are discussed in the literature. In practice, the problems containing integer or discrete variables are usually solved as a continuous problem, and the nearest available discrete value is then selected. Or alternatively all integer and discrete variables are kept fixed and they are not subject to optimization at all. In both cases the result is often quite far from optimum. The reason for taking these approaches is, that it is still commonly considered that no really satisfactory non-linear programming method appear to be available, which is able to handle all integer, discrete and continuous variables and which is also efficient, effective, robust and easy to use. Generally, continuous problems are considered to be easier to solve than discrete ones, suggesting that the presence of any non-continuous variables considerably increases the difficulty of finding a solution.

Another source of difficulties in practical non-linear optimization is handling of constraints. Often multiple constraints are involved, and often the nature of the constraints is non-linear and non-trivial, too.

The feasible solutions may be only a small subset of the design space.

Many different methods have been proposed for solving mixed integer-discrete-continuous, non-linear programming problems. As an example, some engineering design related works were collected in Table 1. Recently, many soft-computing based methods have been under investigations, and were found to be highly potential. However, no single approach seems to be totally satisfying when all the involved aspects are considered. Without pointing to any single method, it can be stated, that all of them suffer from at least one of the following shortcomings: complexity of implementation and usage, lack of flexibility, high computational cost, poor robustness, limited constraint handling capabilities, poor capabilities in finding feasible high quality solutions.

In order to response these demands of practical engineering optimization, a novel approach for solving mixed integer-discrete-continuous, non-linear engineering design optimization problems has been developed based on the recently introduced *Differential Evolution* (DE) algorithm [SP95a]. This article describes the techniques needed to handle boundary constraints as well as those needed to simultaneously deal with several non-linear and non-trivial constraint functions.

Table 1: Some proposed methods for solving engineering design related mixed integer-discrete-continuous, non-linear optimization problems. Comparisons of these methods are reported in [LZ98, LZ99a, LZ99b, LZ99c]. The DE based method described here outperformed all the others. The well-known Sandgren's set of test problems [San90] was used as a test suite.

Mixed Integer-Discrete-Continuous, Non-Linear Optimization		
<i>Reported by</i>	<i>Proposed solution technique</i>	<i>Reference</i>
Sandgren	Branch & Bound using Sequential Quadratic programming	[San90]
Fu, Fenton & Gleghorn	Integer-Discrete-Continuous Non-Linear Programming	[FFG91]
Loh & Papalambros	Sequential Linearization Algorithm	[LP91a, LP91b]
Zhang & Wang	Simulated Annealing	[ZW93]
Chen & Tsao	Genetic Algorithm	[CT93]
Li & Chow	Non-Linear Mixed-discrete Programming	[LC94]
Wu & Chow	Meta-Genetic Algorithm	[WC95]
Lin, Zhang & Wang	Modified Genetic Algorithm	[LZW95]
Thierauf & Cai	Two-level parallel Evolution Strategy	[TC97]
Cao & Wu	Evolutionary Programming	[CW97]
Lampinen & Zelinka	Differential Evolution	This article

2 Mixed Integer-Discrete-Continuous Non-Linear Programming

A mixed integer-discrete-continuous, non-linear programming problem can be expressed as follows:

(1)

Find

$$X = \{x_1, x_2, x_3, \dots, x_n\} = [X^{(i)}, X^{(d)}, X^{(c)}]^T$$

to minimize

$$f(X)$$

subject to constraints

$$g_j(X) \leq 0 \quad j = 1, \dots, m$$

and subject to boundary constraints

$$x_i^{(L)} \leq x_i \leq x_i^{(U)} \quad i = 1, \dots, n$$

where

$$X^{(i)} \in \mathbf{R}^i, \quad X^{(d)} \in \mathbf{R}^d, \quad X^{(c)} \in \mathbf{R}^c$$

$X^{(i)}$, $X^{(d)}$ and $X^{(c)}$ denote feasible subsets of integer, discrete and continuous variables respectively. The above formulation is general and basically the same for all types of variables. Only the structure of the design domain distinguishes one problem from another. However, it is worth to notice here the principal differences between integer and discrete variables. While both integer and discrete variables have a discrete nature, only discrete variables can assume floating-point values. In practice the discrete values of the feasible set are often unevenly spaced. These are the main reasons why integer and discrete variables requires different handling.

3 Differential Evolution

Price and Storn first introduced the Differential Evolution (DE) algorithm a few years ago [SP95a]. DE can be categorized into a class of *floating-point encoded, evolutionary optimization algorithms*. Currently, there are several variants of DE. The particular variant used throughout this investigation is the *DE/rand/1/bin* scheme. This scheme will be discussed here only briefly, since more detailed descriptions are provided in [SP95a, SP97a]. Since the DE algorithm was originally designed to work with continuous variables, the optimization of continuous problems is discussed first. Handling of integer and discrete variables are to be explained later.

Generally, the function to be optimized, f , is of the form:

$$f(X): \mathbf{R}^n \rightarrow \mathbf{R} \quad (2)$$

The optimization target is to minimize the value of this *objective function* $f(X)$,

$$\min(f(X)) \quad (3)$$

by optimizing the values of its parameters:

$$X = (x_1, \dots, x_{n_{param}}) \quad x \in \mathbf{R} \quad (4)$$

where X denotes a vector composed of n_{param} objective function parameters. Usually, the parameters of the objective function are also subject to lower and upper boundary constraints, $x^{(L)}$ and $x^{(U)}$, respectively:

$$x_j^{(L)} \leq x_j \leq x_j^{(U)} \quad j = 1, \dots, n_{param} \quad (5)$$

As with all evolutionary optimization algorithms, DE works with a population of solutions, not with a single solution for the optimization problem. Population P of generation G contains n_{pop} solution vectors called individuals of the population. Each vector represents potential solution for the optimization problem.

$$P^{(G)} = X_i^{(G)} \quad i = 1, \dots, n_{pop}, G = 1, \dots, G_{max} \quad (6)$$

So, the population P of generation G contains n_{pop} individuals each containing n_{param} parameters (*chromosomes* of individuals):

$$P^{(G)} = X_i^{(G)} = x_{i,j}^{(G)} \quad i = 1, \dots, n_{pop}, j = 1, \dots, n_{param} \quad (7)$$

In order to establish a starting point for optimum seeking, the population must be initialized. Often there is no more knowledge available about the location of a global optimum than the boundaries of the problem variables. In this case, a natural way to initialize the population $P^{(0)}$ (initial population) is to seed it with random values within the given boundary constraints:

$$P^{(0)} = x_{i,j}^{(0)} = r_{i,j}(x_j^{(U)} - x_j^{(L)}) + x_j^L \quad i = 1, \dots, n_{pop}, j = 1, \dots, n_{param} \quad (8)$$

where r denotes to a uniformly distributed random value within range $[0.0, 1.0[$.

The population reproduction scheme of DE is different from the other evolutionary algorithms. From the 1st generation forward, the population of the following generation $P^{(G+1)}$ is created in the following way on basis of the current population $P^{(G)}$. First a temporary (trial) population for the subsequent generation, $P^{(G+1)}$, is generated as follows:

(9)

$$x_{i,j}^{(G+1)} = \begin{cases} x_{C_i,j}^{(G)} + F \cdot (x_{A_i,j}^{(G)} - x_{B_i,j}^{(G)}) & \text{if } r_{i,j} \leq C_r \vee j = D_i \\ x_{i,j}^{(G)} & \text{otherwise} \end{cases}$$

where,

$$i = 1, \dots, n_{pop}, j = 1, \dots, n_{param}$$

$$D = 1, \dots, n_{param}$$

$$A = 1, \dots, n_{pop}, B = 1, \dots, n_{pop}, C = 1, \dots, n_{pop}, A_i \neq B_i \neq C_i \neq i$$

$$C_r \in [0,1], F \in [0,2], r \in [0,1[$$

A , B and C are three randomly chosen indexes referring to three randomly chosen individuals of population. They are mutually different from each other and also different from the running index i . New, random, values for A , B and C are assigned for each value of index i (for each individual). A new value for random number r is assigned for each value of index j (for each chromosome).

The index D refers to a randomly chosen chromosome and it is used to ensure that at least one chromosome of each individual vector $X^{(G+1)}$ differs from its counterpart in the previous generation $X^{(G)}$. A new random (integer) value is assigned to D for each value of index i (for each individual).

An illustrative example on how the population of next generation is reproduced on basis of population of current generation is given in Figure 1.

F and C_r are DE control parameters. Both values remain constant during the search process. As well the third control parameter, n_{pop} (population size), remain constant, too. F is a real-valued factor in range $[0.0, 2.0]$ that controls the amplification of differential variations and C_r is a real-valued crossover factor in range $[0.0, 1.0]$ controlling the probability to choose mutated value for x instead of its current value. Generally, both F and C_r affect the convergence velocity and robustness of the search process. Their optimal values are dependent both on objective function, $f(X)$, characteristics and on the population size n_{pop} . Fortunately, DE is not overly sensitive to the values of its control parameters. Usually, suitable values for F , C_r and n_{pop} can be found by trial-and-error after a few tests using different values. Practical advice on how to select control parameters n_{pop} , F and C_r can be found in [SP95a, Sto96a, SP97a, Pri99].

Usually, higher values of n_{pop} and F result in a more robust search but with expense of a lower convergence velocity. Too low values of n_{pop} and F result in premature convergence and/or stagnation before finding the global optima. Typically n_{pop} varies from $2n_{param}$ to $20n_{param}$. Small population size is recommended for unimodal and moderately multimodal problems. However, highly multimodal problems may require n_{pop} higher than $50n_{param}$. According practical experiences F lower than $2/n_{pop}$ or higher than 1.2 are unlikely to work best. Thus, F within range $[(2/n_{pop}), 1.2]$ should be used as initial guess.

Generally a high value of C_r , from 0.9 to 1.0 can be recommended as a starting point unless the objective function is *a priori* known to be separable. Only in case of separable functions or functions with a low degree of *epistasis*, interaction between variables (variable dependent on another), low values of C_r may result in a higher convergence speed. With value $C_r=1.0$ the algorithm is rotationally invariant, which is important when solving real life problems which are rarely separable functions but more often subject to non-linear interactions between the variables. See [Sal96, Pri99] for more details about this subject.

The selection scheme of DE also differs from the other evolutionary algorithms. On basis of the current population $P^{(G)}$ and the temporary population $P^{(G+1)}$, the population of the next generation $P^{(G+1)}$ is created as follows:

(10)

$$X_i^{(G+1)} = \begin{cases} X_i^{(G+1)} & \text{if } f_{cost}(X_i^{(G+1)}) \leq f_{cost}(X_i^{(G)}) \\ X_i^{(G)} & \text{otherwise} \end{cases}$$

Thus, each individual of the temporary (trial) population is compared with its counterpart in the current population. The one with the lower value of cost function $f_{cost}(X)$ (to be minimized) will survive to the population of the next generation. As a result, all the individuals of the next generation are as good or better than their counterparts in the current generation. The interesting point concerning DE's selection scheme is that a trial vector is not compared against all the individuals in the current population, but only against one individual, against its counterpart in the current population.

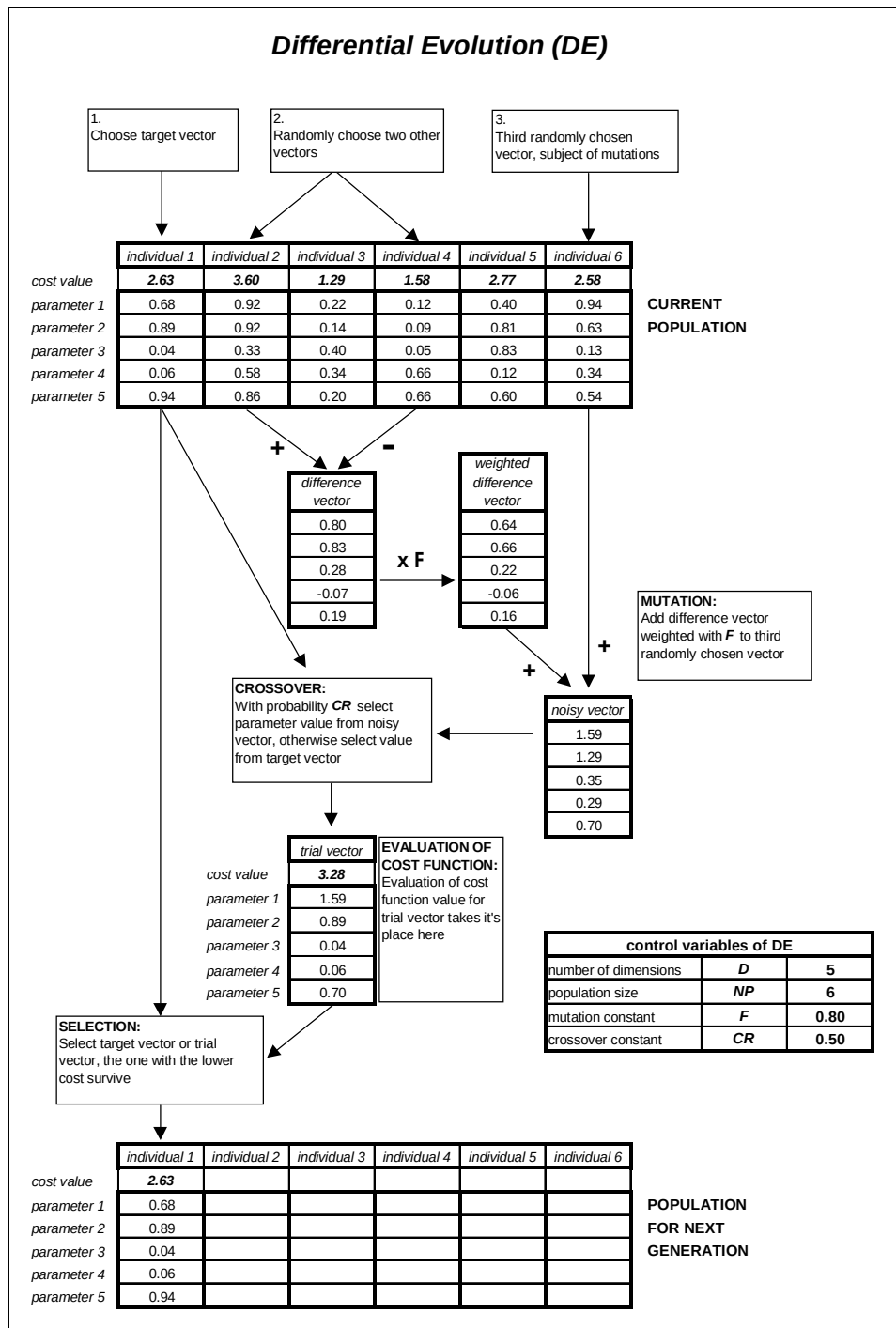


Figure 1. Differential Evolution works directly with the floating-point valued variables of the objective function, not with their (binary) encoding. The functioning of DE is here illustrated in case of a simple objective function $f(X) = x_1 + x_2 + x_3 + x_4 + x_5$.

4 Constraint Handling

4.1 Boundary constraints

It is important to notice that the reproduction operation of DE is able to extend the search outside of the initialized range of the search space (Equation 8 and 9). It is also worthwhile to notice that sometimes this is beneficial property in problems with no boundary constraints because it is possible to find the optimum that is located outside of the initialized range. However, in boundary constrained problems, it is essential to ensure that parameter values lie inside their allowed ranges after reproduction. A simple way to guarantee this is to replace parameter values that violate boundary constraints with random values generated within the feasible range:

(11)

$$x_{i,j}^{(G+1)} = \begin{cases} r_{i,j}(x_j^{(U)} - x_j^{(L)}) + x_j^{(L)} & \text{if } x_{i,j}^{(G+1)} < x_j^{(L)} \vee x_{i,j}^{(G+1)} > x_j^{(U)} \\ x_{i,j}^{(G+1)} & \text{otherwise} \end{cases}$$

where,

$$i = 1, \dots, n_{pop}, j = 1, \dots, n_{param}$$

Another simple method is to reproduce the boundary constraint violating values according Equation 9 as many times as is necessary to satisfy the boundary constraints.

4.2 Constraint functions

A soft-constraint (penalty) approach was applied for handling of the constraint functions. The constraint function introduces a distance measure from the feasible region, but is not used to reject unfeasible solutions, as it is in the case of hard-constraints. One possible soft-constraint approach is to formulate the cost-function as follows:

(12)

$$f_{cost}(X) = (f(X) + a) \cdot \prod_{i=1}^m c_i^{b_i}$$

where

$$c_i = \begin{cases} 1.0 + s_i \cdot g_i(X) & \text{if } g_i(X) > 0 \\ 1 & \text{otherwise} \end{cases}$$

$$s_i \geq 1$$

$$b_i \geq 1$$

$$\min(f(X)) + a > 0$$

The constant, a , is used to ensure that only non-negative values will be assigned to f_{cost} . When the value of a is set high enough, it does not otherwise affect the search process. Constant, s , is used for appropriate scaling of the constraint function value. The exponent, b , modifies the shape of the optimization surface. Generally, higher values of s and b are used when the range of the constraint function, $g(X)$, is expected to be low. Often setting $s=1$ and $b=1$ works satisfactorily and only if one of the constraint functions, $g_i(X)$, remains violated after the optimization run, it will be necessary to use higher values for s_i or/and b_i .

In many real-world engineering optimization problems, the number of constraint functions is relatively high and the constraints are often non-trivial. It is possible that the feasible solutions are only a small subset of the search space. Feasible solutions may also be divided into separated islands around the search space. Furthermore, the user may easily define totally conflicting constraints so that no feasible solutions exist at all. For example, if two or more constraints conflict, so that no feasible solution exists, DE is still able to find the nearest feasible solution. In the case of non-trivial constraints, the user is often able to judge which of the constraints are conflicting on the basis of the nearest feasible solution. It is then possible to reformulate the cost-function or reconsider the problem setting itself to resolve the conflict.

Another benefit of the soft-constraint approach is that the search space remains continuous. Multiple hard constraints often split the search space into many separated islands of feasible solutions. This discontinuity introduces stalling points for some genetic searches and also raises the possibility of new, locally optimal areas near the island borders. For these reasons a soft-constraint approach is considered essential. It should be mentioned that many traditional optimization methods are only able to handle hard-constraints. For evolutionary optimization, the soft-constraint approach was found to be a natural approach.

5 Handling of Integer and Discrete Variables

In its canonical form, the Differential Evolution algorithm is only capable of handling continuous variables. Extending it for optimization of integer variables, however, is rather easy. Only a couple of simple modifications are required. First, for evaluation of cost-function, integer values should be used. Despite of that the DE itself may still work internally with continuous floating-point values. Thus,

(13)

$$f_{\text{cost}}(y_i) \quad i = 1, \dots, n_{\text{param}}$$

where

$$y_i = \begin{cases} x_i & \text{for continuous variables} \\ \text{INT}(x_i) & \text{for integer variables} \end{cases}$$

$$x_i \in X$$

$\text{INT}()$ is a function for converting a real value to an integer value by truncation. Truncation is performed here only for purposes of cost-function value evaluation. Truncated values are not elsewhere assigned. Thus, DE works with a population of continuous variables regardless of the corresponding object variable type. This is essential for maintaining the diversity of the population and the robustness of the algorithm. Second, in case of integer variable, instead of Equation 8, the population should be initialized as follows:

(14)

$$P^{(0)} = x_{i,j}^{(0)} = r_{i,j}(x_j^{(U)} - x_j^{(L)} + 1) + x_j^{(L)} \quad i = 1, \dots, n_{\text{pop}} \quad , \quad j = 1, \dots, n_{\text{param}}$$

Additionally, instead of Equation 11, the boundary constraint handling for integer variables should be performed as follows:

(15)

$$x_{i,j}^{(G+1)} = \begin{cases} r_{i,j}(x_j^{(U)} - x_j^{(L)} + 1) + x_j^{(L)} & \text{if } \text{INT}(x_{i,j}^{(G+1)}) < x_j^{(L)} \quad \vee \quad \text{INT}(x_{i,j}^{(G+1)}) > x_j^{(U)} \\ x_{i,j}^{(G+1)} & \text{otherwise} \end{cases}$$

where,

$$i = 1, \dots, n_{\text{pop}} \quad , \quad j = 1, \dots, n_{\text{param}}$$

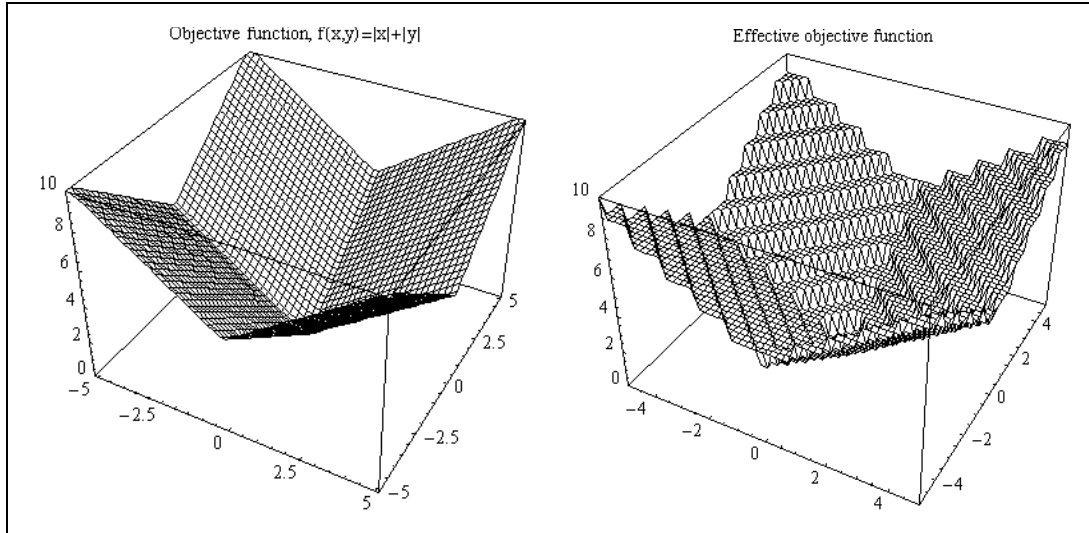


Figure 2. The suggested method for handling integer and discrete variables illustrated in case of a simple 2D objective function $f(x,y)=|x|+|y|$. Continuous variable case (left) and discrete (right) case in which only integer values are allowed. In discrete case the optimization algorithm still works with a continuous objective function surface, but the surface is modified according to Equations 13-16 as illustrated. A discrete problem is actually converted to a corresponding continuous one.

Discrete values can also be handled in a straightforward manner. Suppose that the subset of discrete variables, $X^{(d)}$, contains l elements that can be assigned to variable x :

(16)

$$X^{(d)} = x_i^{(d)} \quad i = 1, \dots, l$$

where

$$x_i^{(d)} < x_{i+1}^{(d)}$$

Instead of the discrete value x_i itself, we may assign its index, i , to x . Now the discrete variable can be handled as an integer variable that is boundary constrained to range $1, \dots, l$. To evaluate of the objective function, the discrete value, x_i , is used instead of its index i . In other words, instead of optimizing the value of the discrete variable directly, we optimize the value of its index i . Only during evaluation is the indicated discrete value used. Once the discrete problem has been converted into an integer one, the previously described methods for handling integer variables can be applied (Equations 13–15).

6 Usage of DE in Practice

Practically oriented numerical examples on using DE for solving mixed discrete-integer-continuous non-linear optimization problems and comparisons with other methods can be found in [LZ98, LZ99a, LZ99b, LZ99c]. Also a high number of continuous non-linear optimization problems, including practical applications, have been solved until now. For example DE have been applied for optimizing and learning Neural Networks [ML97, ZL99b], solving inverse fractal problems [ZL99a], non-linear process control [SBN99], designing digital filters [Sto96c, Sto99b], just mentioning few. An application of DE in the field of prediction is reported elsewhere in this proceedings. For a comprehensive updated bibliography of DE, see [Lam99].

7 Conclusions

The described method is relatively simple, easy to implement and easy to use. Despite of that, it is capable of optimizing all integer, discrete and continuous variables and capable of handling non-linear objective functions with multiple non-trivial constraints.

A soft-constraint (penalty) approach is applied for handling of constraint functions. Some optimization methods require a feasible initial solution as a starting point for a search. Preferably, this solution should be rather close to a global optimum to ensure convergence to it instead of a local optimum. If non-trivial constraints are imposed, it may be difficult or impossible to provide a feasible initial solution. The efficiency, effectiveness and robustness of many methods are often highly dependent on the quality of the starting point. The combination of DE with the soft-constraint approach does not require any initial solution, but it can still take advantage of a high quality initial solution if one is available. For example, this initial solution can be used for initialization of the population in order to establish an initial population that is biased towards the feasible region of the search space.

If there are no feasible solutions in the search space, as is the case for totally conflicting constraints, DE with the soft-constraint approach is still able to find the nearest feasible solution. This is important in practical engineering optimization because often many non-trivial constraints are involved.

The described approach was targeted to fill the gap in the field of mixed discrete-integer-continuous optimization, where no really satisfactory methods appeared to be available. Despite being in its infancy, the described approach has great potential to become a widely used, multipurpose optimization tool for solving a broad range of practical engineering optimization problems.

The stochastic *Differential Evolution* algorithm is undoubtedly one of the most promising novel methods for non-linear optimization. The DE algorithm can be applied generally and it works with minimum assumptions with respect to the objective function. The algorithm requires only the value of objective function for guidance of its optimum seeking. No derivatives or other auxiliary information. Including the algorithm extensions discussed in this article, DE can be applied for a wide range of optimization problems, which the practitioners in the field of modern prediction would like to solve.

References

- [CW97] Cao, Y. J. and Wu, Q. H. (1997). *Mechanical design optimization by mixed-variable evolutionary programming*. Proceedings of the 1997 IEEE Conference on Evolutionary Computation, pp. 443–446.
- [CT93] Chen, J. L. and Tsao, Y. C. (1993). *Optimal design of machine elements using genetic algorithms*. Journal of the Chinese Society of Mechanical Engineers, 14(2):193–199, 1993.
- [FFG91] Fu, J.-F., Fenton, R. G. and Cleghorn, W. L. (1991). *A mixed integer-discrete-continuous programming method and its application to engineering design optimization*. Engineering Optimization, 17(4):263–280, 1991. ISSN 0305-2154
- [Lam99] Jouni Lampinen (1999). *Bibliography of Differential Evolution 1995–1999*. Available via Internet, <http://ft3.zlin.vutbr.cz/zelinka/DE/Biblio/Biblio.htm>
- [LZ98] Jouni Lampinen – Ivan Zelinka (1998). *Mechanical Engineering Design Optimization by Differential Evolution*. Internal Report. University of Vaasa, department of information technology and production economics. (available from authors on request)
- [LZ99a] Jouni Lampinen – Ivan Zelinka (to appear). *Mechanical Engineering Design Optimization by Differential Evolution*. In: *Unknown Bookname* (new methods for optimization). Mc-Graw-Hill, New York.
- [LZ99b] Jouni Lampinen – Ivan Zelinka (1999). *Mixed Integer-Discrete-Continuous Optimization By Differential Evolution, Part 1: the optimization method*. In: Ošmera, Pavel (ed.) (1999). Proceedings of MENDEL'99, 5th Inter-

national Mendel Conference on Soft Computing, June 9.–12. 1999, Brno, Czech Republic. Brno University of Technology, Faculty of Mechanical Engineering, Institute of Automation and Computer Science, Brno (Czech Republic), pp. 71–76. ISBN 80-214-1131-7

[LZ99c] Jouni Lampinen – Ivan Zelinka (1999). *Mixed Integer-Discrete-Continuous Optimization By Differential Evolution, Part 2: a practical example*. In: Ošmera, Pavel (ed.) (1999). *Proceedings of MENDEL'99, 5th International Mendel Conference on Soft Computing*, June 9.–12. 1999, Brno, Czech Republic. Brno University of Technology, Faculty of Mechanical Engineering, Institute of Automation and Computer Science, Brno (Czech Republic), pp. 77–81. ISBN 80-214-1131-7

[LC94] Li, H.-L. and Chou, C.-T. (1994). *A global approach for nonlinear mixed discrete programming in design optimization*. *Engineering Optimization*, 22():109–122, 1994.

[LZW95] Lin, Shui-Shun, Zhang, Chun and Wang, Hsu-Pin (1993). *On mixed-discrete nonlinear optimization problems: A comparative study*. *Engineering Optimization*, 23(4):287–300, 1995. ISSN 0305-215X

[LP91a] Loh, Han Tong and Papalambros, P. Y. (1991). *A sequential linearization approach for solving mixed-discrete nonlinear design optimization problems*. *Transactions of the ASME, Journal of Mechanical Design*, 113(3):325–334, September 1991.

[LP91b] Loh, Han Tong and Papalambros, P. Y. (1991). *Computational implementation and tests of a sequential linearization algorithm for mixed-discrete nonlinear design optimization*. *Transactions of the ASME, Journal of Mechanical Design*, 113(3):335–345, September 1991.

[ML97] Masters, Timothy and Land, Walker (1997). *A new training algorithm for the general regression neural network*. 1997 IEEE International Conference on Systems, Man, and Cybernetics, Computational Cybernetics and Simulation., Volume: 3, pp. 1990–1994.

[Pri99] Price, Kenneth V. (to appear). *An Introduction to Differential Evolution*. In: *Unknown Bookname* (new methods for optimization). Mc-Graw-Hill, New York.

[Sal96] Salomon, Ralf (1996). *Reevaluating Genetic Algorithm Performance under Coordinate Rotation of Benchmark Functions; A survey of some theoretical and practical aspects of genetic algorithms*. *BioSystems*, 39(3):263–278, 1996, Elsevier Science.

[SBN99] K. K. N. Sastry, L. Behera, I. J. Nagrath (1999). *Differential Evolution Based Fuzzy Logic Controller for Nonlinear Process Control*. *Fundamenta Informaticae* 37(1–2):121–136, January 1999.

[Sto96a] Storn, Rainer (1996). *On the usage of differential evolution for function optimization*. *NAFIPS 1996*, Berkeley, pp. 519 - 523.

[Sto96c] Storn, Rainer (1996). *Differential evolution design of an IIR-filter*. *Proceedings of IEEE International Conference on Evolutionary Computation*, 20-22 May 1996, pp. 268–273. ISBN: 0-7803-2902-3

[Sto99b] Storn, Rainer (to appear). *Designing Digital Filters with Differential Evolution*. In: *Unknown Bookname* (new methods for optimization). Mc-Graw-Hill, New York.

[SP95a] Storn, Rainer and Price, Kenneth (1995). *Differential evolution - a simple and efficient adaptive scheme for global optimization over continuous spaces*. Technical Report TR-95-012, ICSI, March 1995. (Available via ftp from ftp.icsi.berkeley.edu/pub/techreports/1995/tr-95-012.ps.Z)

[SP97a] Storn, Rainer and Price, Kenneth (1997). *Differential Evolution – A simple evolution strategy for fast optimization*. *Dr. Dobbs's Journal*, April 97, pp. 18–24 and p. 78.

[San90] Sandgren, E. (1990). *Nonlinear integer and discrete programming in mechanical design optimization*. *Transactions of the ASME, Journal of Mechanical Design*, 112(2):223–229, June 1990. ISSN 0738-0666

[TC97] Thierauf, G. and Cai, J. (1997). *Evolution strategies – parallelisation and application in engineering optimization*. In B.H.V. Topping (ed.) (1997). *Parallel and distributed processing for computational mechanics*. Saxe-Coburg Publications, Edinburgh (Scotland). ISBN 1-874672-03-2

[WC95] Wu, S.-J. and Chow, P.-T. (1995). *Genetic algorithms for nonlinear mixed discrete-integer optimization problems via meta-genetic parameter optimization*. *Engineering Optimization*, 24(2):137–159, 1995. ISSN 0305-215X

[Zel98] Ivan Zelinka (ed.) (1998). *Proceedings of Nostradamus '98, 1st Prediction Workshop*, September 22.–23. 1998, Zlin, Czech Republic. Brno University of Technology, Faculty of Technology (Zlin), Department of Automatic Control, Zlin (Czech Republic). ISBN 80-214-1222-4

[ZL99a] Ivan Zelinka – Jouni Lampinen (1999). *Inverse Fractal Problem by Means of Evolutionary Algorithms*. In: Ošmera, Pavel (ed.) (1999). *Proceedings of MENDEL'99, 5th International Mendel Conference on Soft Computing*, June 9.–12. 1999, Brno, Czech Republic. Brno University of Technology, Faculty of Mechanical Engineering, Institute of Automation and Computer Science, Brno (Czech Republic), pp. 430–435. ISBN 80-214-1131-7

[ZL99b] Ivan Zelinka – Jouni Lampinen (1999). *DELA – an Evolutionary Learning Algorithms for Neural Networks*. In: Ošmera, Pavel (ed.) (1999). *Proceedings of MENDEL'99, 5th International Mendel Conference on Soft Computing*, June 9.–12. 1999, Brno, Czech Republic. Brno University of Technology, Faculty of Mechanical Engineering, Institute of Automation and Computer Science, Brno (Czech Republic), pp. 410–414. ISBN 80-214-1131-7

[ZW93] Zhang, Chun and Wang, Hsu-Pin (1993). *Mixed-discrete nonlinear optimization with simulated annealing*. *Engineering Optimization*, 21(4):277–291, 1993. ISSN 0305-215X

Reference data for this article:

[LZ99d] Jouni Lampinen – Ivan Zelinka (1999). *Mixed Variable Non-Linear Optimization by Differential Evolution*. In: Zelinka, Ivan (ed.) (1999). *Proceedings of Nostradamus'99, 2nd International Prediction Conference, October 7.–8. 1999, Zlin, Czech Republic*. Technical University of Brno, Faculty of Technology Zlin, Department of Automatic Control, Zlin (Czech Republic), pp. 45–55. ISBN 80-214-1424-3.